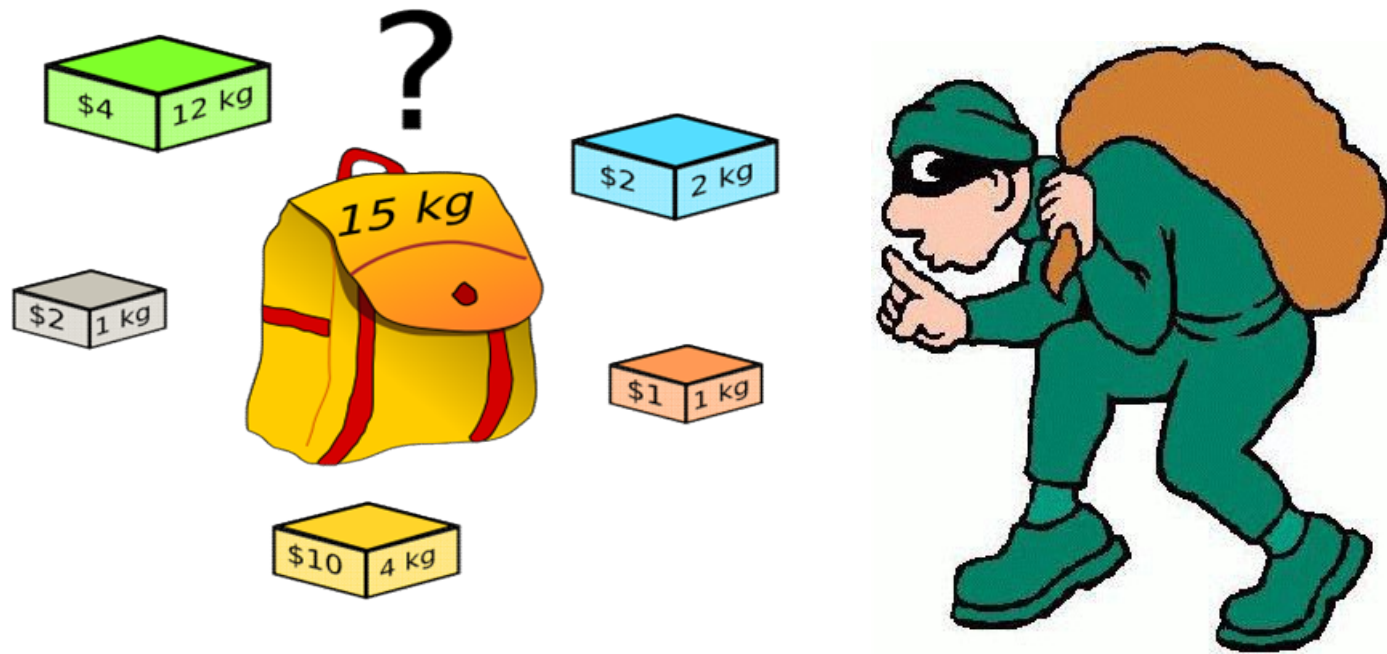


Fractional Knapsack Problem

A thief considers taking W Kg of loot.

The loot is in the form of n items, each with weight w_i and value v_i .

Any amount of an item can be put in the knapsack ,weight limit W is not exceeded.



Fractional Knapsack Problem

Item	Value	Weight
1	4	12
2	2	1
3	10	4
4	1	1
5	2	2

Let's select items which gives me highest value

Item 3 Profit =10 Remaining weight is $15-4 = 11$

Item 1 Profit = $10+4*(11/12) = 10 + 3.66 = 13.66$

Remaining weight is $11 -11 =0$



Fractional Knapsack Problem

Item	Value	Weight
1	4	12
2	2	1
3	10	4
4	1	1
5	2	2

Let's select items with lowest weight that keeps knapsack empty for long period

Item 2 Profit = 2 Remaining weight is $15 - 1 = 14$

Item 4 Profit = $1 + 2$ Remaining weight is $14 - 1 = 13$

Item 5 Profit = $1 + 2 + 2$ Remaining weight is $13 - 2 = 11$

Item 3 Profit = $1 + 2 + 2 + 10 = 15$
Remaining weight is $11 - 4 = 7$

Item 1 Profit = $15 + 4 * (7/12) = 15 + 2.33 = 17.33$
Remaining weight is $7 - 7 = 0$



Fractional Knapsack Problem

Item	Value	Weight	Value/weight
1	4	12	$4/12 = 0.33$
2	2	1	$2/1 = 2$
3	10	4	$10/4 = 2.5$
4	1	1	$1/1 = 1$
5	2	2	$2/2 = 1$

Let's select items with highest profit per unit weight

Item 3 Profit = 10 Remaining weight is $15 - 4 = 11$

Item 2 Profit = $10 + 2$ Remaining weight is $11 - 1 = 10$

Item 5 Profit = $10 + 2 + 2$ Remaining weight is $10 - 2 = 8$

Item 4 Profit = $10 + 2 + 2 + 1 = 15$
Remaining weight is $8 - 1 = 7$

Item 1 Profit = $15 + 4 \cdot (7/12) = 15 + 2.33 = 17.33$
Remaining weight is $7 - 7 = 0$



Greedyknapsack($v[1..n]$, $w[1..n]$, W)

```
For i= 1 to n  
    x[i]=0  
    ratio[i]= v[i]/w[i]  
end  
Weight=0  
Profit=0
```

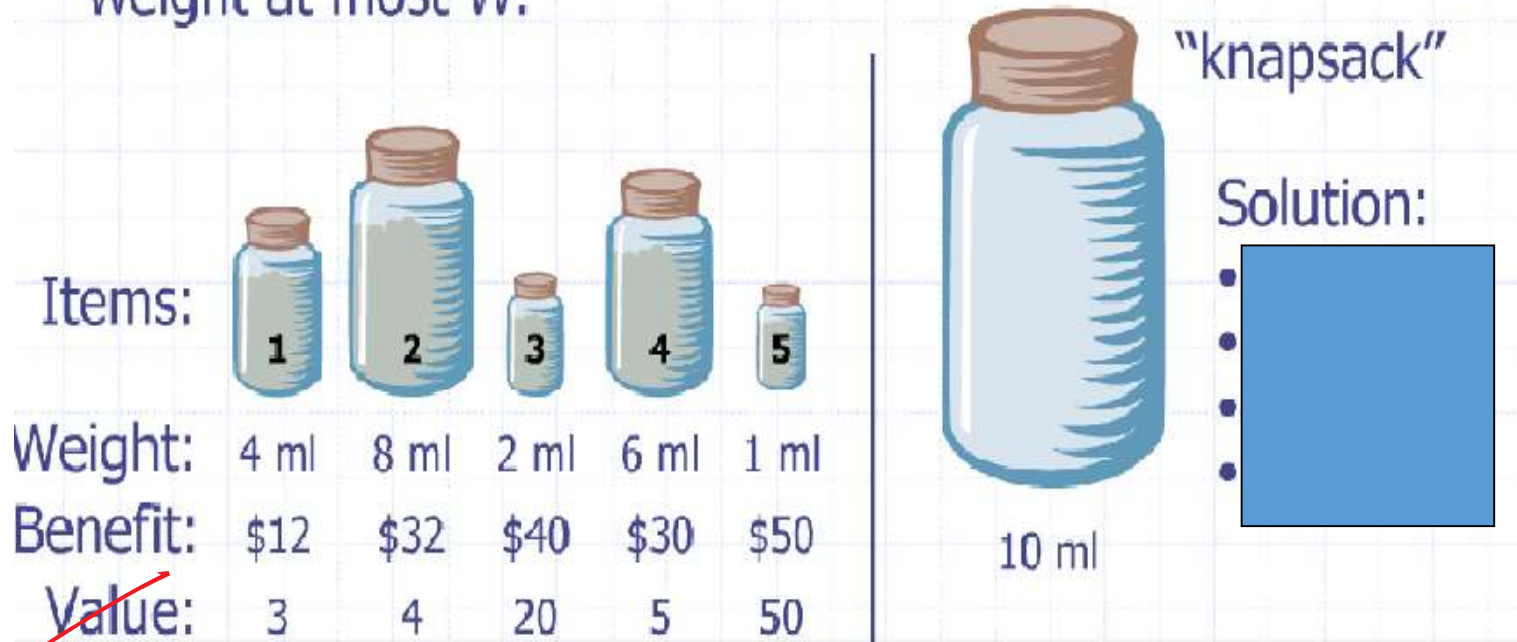
```
While (weight < W)  
    i= select item from highest ratio to lowest ratio in order  
    if (weight+w[i] <= W )then  
        x[i]=1  
        profit=profit +x[i]*v[i]  
        weight= weight + w[i]  
    else  
        x[i]= (W-weight)/w[i]  
        profit=profit +x[i]*v[i]  
        weight= W.  
    end  
end  
Return profit.
```

Greedy Algorithm for Fractional Knapsack problem

- Fractional knapsack can be solvable by the greedy strategy
 - Compute the value per unit weight v_i/w_i for each item
 - Obeying a greedy strategy, take as much as possible of the item with the greatest value per unit weight.
 - If the supply of that item is exhausted and there is still more room, take as much as possible of the item with the next value per unit weight, and so forth until there is no more room
 - $O(n \lg n)$ (we need to sort the items by value per unit)

Fractional Knapsack Problem (2)

- ◆ Given: A set S of n items, with each item i having
 - b_i - a positive benefit
 - w_i - a positive weight
- ◆ Goal: Choose items with maximum total benefit but with weight at most W .



Ratio

Fractional Knapsack Problem (3)

Example 4.1 Consider the following instance of the knapsack problem:
 $n = 3, m = 20, (p_1, p_2, p_3) = (25, 24, 15)$, and $(w_1, w_2, w_3) = (18, 15, 10)$.

Fractional Knapsack Problem (3)

Example 4.1 Consider the following instance of the knapsack problem:
 $n = 3, m = 20, (p_1, p_2, p_3) = (25, 24, 15)$, and $(w_1, w_2, w_3) = (18, 15, 10)$.
Four feasible solutions are:

	(x_1, x_2, x_3)	$\sum w_i x_i$	$\sum p_i x_i$
1.	$(1/2, 1/3, 1/4)$	16.5	24.25
2.	$(1, 2/15, 0)$	20	28.2
3.	$(0, 2/3, 1)$	20	31
4.	$(0, 1, 1/2)$	20	31.5

Change-Making Problem

Given unlimited amounts of coins of denominations $d_1 > \dots > d_m$,
give change for amount n with the least number of coins

Example: $d_1 = 25$, $d_2 = 10$, $d_3 = 5$, $d_4 = 1$ and $n = 48$

Greedy solution: $\langle 1, 2, 0, 3 \rangle$

Greedy solution is

- optimal for any amount and “normal” set of denominations

Change-Making Problem - Algorithm

```
{  
int i, count = 0;  
  
for(i = 0; i < size; i++)  
{ //take as much from coins[i]  
    while(value >= coins[i])  
    { //after taking the coin, reduce the value.  
        value -= coins[i];  
        ans[count] = coins[i];  
        count++;  
    }  
    if(value == 0) break;  
}  
return count;  
}
```

coin[] = {25,20,10,5}
value = 70

Change-Making Problem

Given unlimited amounts of coins of denominations $d_1 > \dots > d_m$,
give change for amount n with the least number of coins

Example: $d_1 = 25$, $d_2 = 10$, $d_3 = 5$, $d_4 = 1$ and $n = 48$

Greedy solution: $\langle 1, 2, 0, 3 \rangle$

Greedy solution is

- optimal for any amount and “normal” set of denominations
- **may not be** optimal for arbitrary coin denominations

For example, $d_1 = 25$, $d_2 = 10$, $d_3 = 1$, and $n = 30$